

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e Multimédia
Infraestruturas Computacionais Distribuídas

RELATÓRIO TÉCNICO PONTOS E CAIXAS EM ARQUITETURA CLIENTE-SERVIDOR PARTE 1

Alunos:

Rafael Saraiva, 47484

Mariana Almeida, 49749

Turma: 42D

Docente:

Prof. Cedric Grueau

26 de Abril de 2026

ÍNDICE

1. INTRODUÇÃO	3
2. DESENVOLVIMENTO	4
2.1 Arquitetura Geral.....	4
2.2 Descrição dos Nós da Arquitetura.....	4
2.2.1 Cliente	4
2.2.2 Servidor	5
2.2.3 Sistema de Persistência (XML/XSD)	5
2.3 Protocolos de Comunicação.....	5
2.3.1 Protocolo de Transporte	5
2.3.2 Protocolo de Aplicação.....	6
2.3.3 Porta de Comunicação	7
2.4 Fluxo de Execução do Sistema	7
2.4.1 Exemplo: Registo	7
2.4.2 Exemplo: Criação de Jogo	8
2.4.3 Exemplo: Realizar Jogada	8
2.4.4 Exemplo: Finalização do Jogo.....	9
3. CONCLUSÃO	10
3.1 Vantagens da Solução	10
3.2 Desvantagens e Limitações.....	10
4. BIBLIOGRAFIA.....	10

1. INTRODUÇÃO

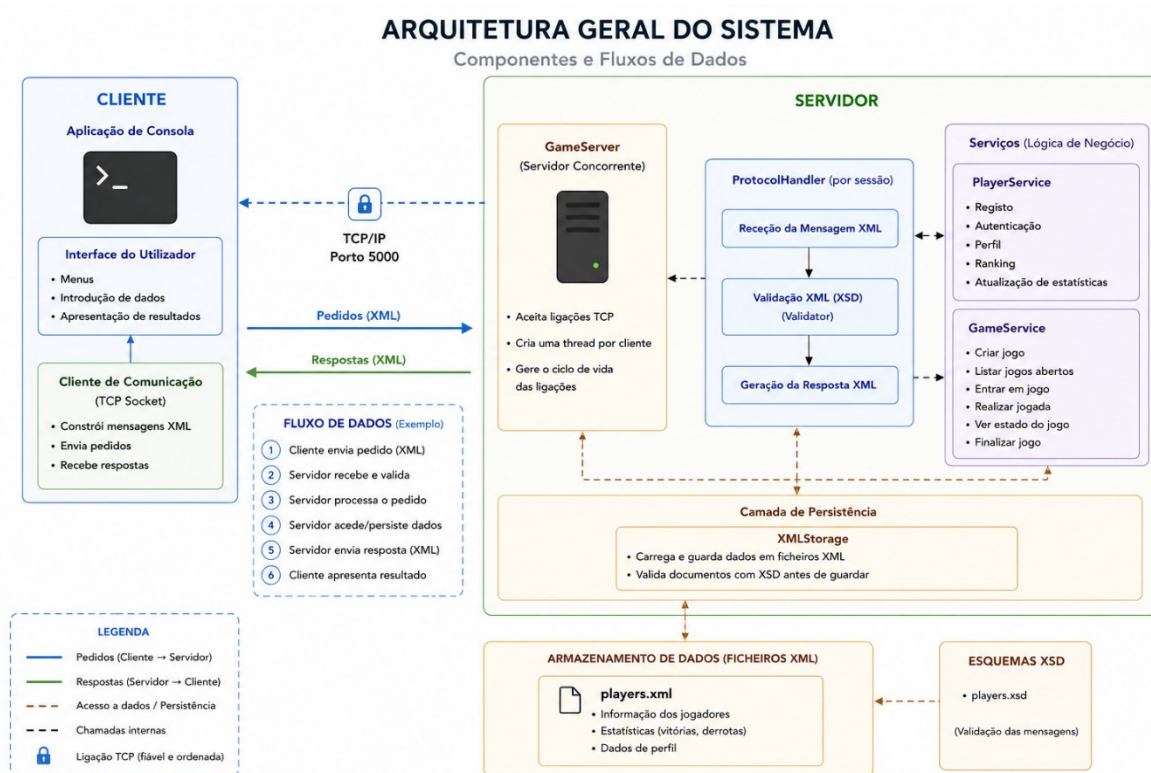
No âmbito da unidade curricular de Infraestruturas Computacionais Distribuídas, foi proposta a implementação de uma infraestrutura distribuída capaz de suportar partidas do jogo Pontos e Caixas entre utilizadores ligados através de uma rede de computadores.

A solução desenvolvida baseia-se numa arquitetura cliente-servidor, onde um servidor central gere toda a lógica de negócio e múltiplos clientes comunicam com este através de ligações TCP. A troca de informação é realizada através de mensagens XML, cuja estrutura é validada recorrendo a esquemas XSD.

O principal objetivo do projeto consistiu em aplicar conceitos relacionados com comunicação distribuída, protocolos de aplicação, persistência de dados e concorrência, desenvolvendo simultaneamente uma solução funcional para o jogo proposto.

2. DESENVOLVIMENTO

2.1 Arquitetura Geral



2.2 Descrição dos Nós da Arquitetura

2.2.1 Cliente

O cliente foi desenvolvido em Java sob a forma de uma aplicação de consola.

Este componente é responsável por:

- Apresentar menus ao utilizador;
- Recolher dados introduzidos pelo jogador;
- Construir mensagens XML;
- Estabelecer ligações TCP ao servidor;
- Receber e interpretar respostas XML.

O cliente pode ser considerado um fat client, uma vez que realiza alguma validação local antes do envio das mensagens para o servidor, nomeadamente se o utilizador está autenticado.

2.2.2 Servidor

O servidor foi implementado através da classe `GameServer`. Trata-se de um servidor concorrente, uma vez que cada ligação recebida é tratada numa thread independente.

As suas principais responsabilidades são:

- Aceitar ligações TCP;
- Receber mensagens XML;
- Validar mensagens através de XSD;
- Executar operações sobre jogadores e jogos;
- Atualizar a persistência;
- Devolver respostas XML aos clientes.

A concorrência permite que vários jogadores utilizem o sistema simultaneamente.

2.2.3 Sistema de Persistência (XML/XSD)

A persistência é assegurada através de um ficheiro XML armazenado localmente: `players.xml`.

Para cada jogador são armazenados:

- Nickname;
- Password;
- Nome completo;
- Nacionalidade;
- Idade;
- Fotografia;
- Cor favorita;
- Número de vitórias;
- Número de derrotas;
- Tempo total de jogo.

O documento XML é validado através de um schema XSD (`players.xsd`) sempre que um novo player é registado. Esta abordagem permite:

- Garantir a integridade dos dados;
- Evitar estruturas inválidas;
- Facilitar a deteção de erros.

2.3 Protocolos de Comunicação

2.3.1 Protocolo de Transporte

A comunicação utiliza o protocolo TCP (Transmission Control Protocol).

A escolha deste protocolo deve-se às suas características:

- Fiabilidade;
- Entrega ordenada das mensagens;
- Detecção de erros;
- Controlo de fluxo.

Todas as operações do sistema requerem consistência dos dados, tornando o TCP a solução mais adequada.

2.3.2 Protocolo de Aplicação

A comunicação entre cliente e servidor é realizada através de mensagens XML.

Cada operação possui uma mensagem de pedido e uma mensagem de resposta associadas.

O protocolo suporta as seguintes operações:

- Register
- Login
- Open Games
- Create Game
- Join Game
- Play Move
- Ranking
- Logout

```
<login><nickname>rafa</nickname><password>abc</password></login>  
  
<response>  
  <status>OK</status>  
  <message>Login efetuado.</message>  
</response>
```

```
<?xml version="1.0" encoding="UTF-8"?>  
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="login">  
    <xs:complexType>  
      <xs:sequence>  
        <xs:element name="nickname" type="xs:string"/>  
        <xs:element name="password" type="xs:string"/>  
      </xs:sequence>  
    </xs:complexType>  
  </xs:element>  
</xs:schema>
```

Cada mensagem representa uma ação concreta do sistema. O servidor interpreta o elemento raiz do XML para determinar qual a operação pretendida. A validação ocorre antes do processamento da lógica de negócio.

A comunicação segue um modelo síncrono:

1. Cliente envia pedido;
2. Servidor valida;
3. Servidor processa;
4. Servidor responde;
5. Cliente apresenta resultado.

Só após receber a resposta é iniciado um novo pedido.

2.3.3 Porta de Comunicação

O servidor encontra-se configurado para utilizar, por defeito, o porto 5000.

2.4 Fluxo de Execução do Sistema

O funcionamento do sistema pode ser descrito através da sequência de operações realizadas entre cliente e servidor. Este fluxo permite compreender como os diferentes componentes interagem para processar um pedido.

2.4.1 Exemplo: Registo

1. O utilizador introduz os seus dados.
2. O cliente gera uma mensagem XML de registo.
3. A mensagem é enviada para o servidor.
4. O servidor valida o XML.
5. O jogador é criado.
6. A informação é armazenada em players.xml.
7. O servidor devolve uma resposta de sucesso.

```
=== Pontos e Caixas - Cliente TCP ===
1. Registrar
2. Login
3. Criar jogo
4. Listar jogos abertos
5. Entrar num jogo
6. Jogar
7. Estado do jogo
8. Ranking
0. Sair
> 1
Nickname: exemplo
Password: exemplo123
Nome completo: Senhor Exemplo
Nacionalidade: PT
Idade: 30
Fotografia: null.png
```

```
<player>
  <nickname>exemplo</nickname>
  <password>f31b6c2baaac3cae888d2d064a5bfa74fbaf72a8b98d01a3e9ddc41d83f93ad0</password>
  <fullName>Senhor Exemplo</fullName>
  <nationality>PT</nationality>
  <age>30</age>
  <photo>null.png</photo>
  <favoriteColor>#f7f7f7</favoriteColor>
  <victories>0</victories>
  <defeats>0</defeats>
  <totalGameTimeMillis>0</totalGameTimeMillis>
</player>
```

```
<response>
  <status>OK</status>
  <message>Jogador registado.</message>
</response>
```

2.4.2 Exemplo: Criação de Jogo

1. Um jogador autenticado solicita a criação de um jogo.
2. O pedido é enviado para o servidor.
3. O servidor cria uma nova partida.
4. O identificador do jogo é devolvido ao cliente.

```
=== Pontos e Caixas - Cliente TCP ===
1. Registrar
2. Login
3. Criar jogo
4. Listar jogos abertos
5. Entrar num jogo
6. Jogar
7. Estado do jogo
8. Ranking
0. Sair
> 3
Linhas: 3
Colunas: 3

<createGameResponse>
  <status>OK</status>
  <gameId>85647923</gameId>
</createGameResponse>
```

2.4.3 Exemplo: Realizar Jogada

1. O jogador envia a jogada.
2. O servidor verifica se a jogada é válida.
3. É verificado o fecho de caixas.
4. São atribuídos pontos quando aplicável.
5. O estado do jogo é atualizado.
6. É enviada uma resposta atualizada ao cliente.

```
> 6
Game ID: 85647923
Row: 1
Col: 1
Direção H/V: H

<moveResponse>
  <status>OK</status>
  <message>Jogada aceite.</message>
  <boxesClosed>0</boxesClosed>
  <nextTurn>exemplo</nextTurn>
</moveResponse>
```

```
<gameStateResponse>
  <status>OK</status>
  <gameId>85647923</gameId>
  <rows>3</rows>
  <cols>3</cols>
  <player1>exemplo</player1>
  <player2>rafa</player2>
  <currentTurn>exemplo</currentTurn>
  <gameStatus>RUNNING</gameStatus>
  <score1>0</score1>
  <score2>0</score2>
  <winner></winner>
  <board>
```

```
. . .
. . .
. . .
```

2.4.4 Exemplo: Finalização do Jogo

Quando todas as linhas do tabuleiro forem preenchidas:

1. O vencedor é determinado.
2. São atualizadas as estatísticas dos jogadores.
3. São registadas vitórias e derrotas.
4. O jogo é marcado como concluído.

3. CONCLUSÃO

O sistema desenvolvido cumpre os requisitos definidos para a primeira parte do trabalho, permitindo a realização de partidas de Pontos e Caixas através de uma arquitetura distribuída baseada em TCP.

A solução implementada demonstra a aplicação prática dos conceitos fundamentais abordados na unidade curricular, nomeadamente comunicação cliente-servidor, concorrência, persistência de dados e definição de protocolos de aplicação.

3.1 Vantagens da Solução

- Arquitetura simples e modular;
- Utilização de TCP para comunicações fiáveis;
- Servidor concorrente;
- Persistência de dados em XML;
- Validação automática através de XSD;
- Facilidade de manutenção e extensão.

3.2 Desvantagens e Limitações

- Dependência de um único servidor;
- Persistência baseada em ficheiros XML pouco adequada para grandes volumes de dados;
- Ausência de mecanismos avançados de recuperação após falha.

4. BIBLIOGRAFIA

Oracle. (2024). Java Servlet technology documentation.
<https://docs.oracle.com/javaee/7/tutorial/servlets.htm>

Apache Software Foundation. (2024). Apache Tomcat documentation.
<https://tomcat.apache.org/tomcat-9.0-doc/>

World Wide Web Consortium (W3C). (2008). Extensible Markup Language (XML) 1.0 (Fifth Edition). <https://www.w3.org/TR/xml/>

World Wide Web Consortium (W3C). (2012). XML Schema Definition Language (XSD) 1.1. <https://www.w3.org/TR/xmlschema11-1/>

Instituto Superior de Engenharia de Lisboa. (2026). Enunciado do Trabalho Prático — Infraestruturas Computacionais Distribuídas.